CSI473 Project Design Report

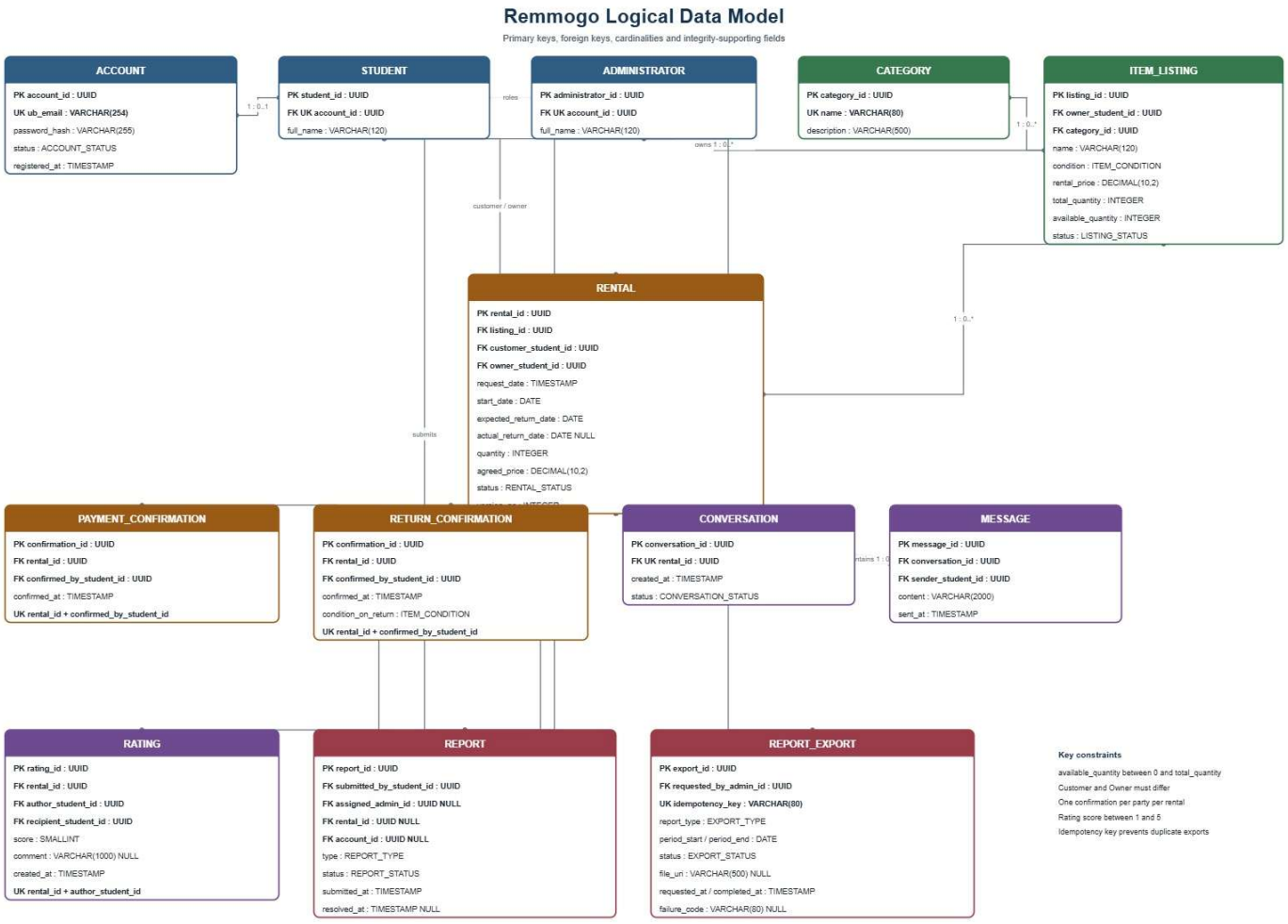
RREMOGO - Team 04

Field	Complete
Assignment/phase	Phase 2
Approved project title	Remmogo
Approval decision/date	17/08/26
Team number	4
Members and IDs	Rumbidzai Zanele Magweregwede 202208048 Keoikantse Ronald Tumisang 201502162 Kgotla Osiame Mogaetsho 202005511 Omatla Tendai Manyanda 202004949 Leander Van der Walt 202307966
Repository URL	https://github.com/LeeLeeGMOSPhipidip/CSI473-Project
Submission tag	
Date	30/10/2026

1. Data, API/service and deployment design

Logical data model

The system stores information about students, listings, rentals, payments, returns, messages, ratings and reports. The main entities are:



Remmogo API Contract

1. Purpose and traceability

This contract defines the operation used by a Remmogo Administrator to generate a printable PDF report of rental transactions. It supports FR-13 Account Management and Reporting, the Generate reports administrator use case, BR-08 Administrative Reporting, and QS-05 Administrative Report Generation. Under QS-05, a semester report containing more than 500 records must be ready within 10 seconds.

The PDF rendering application is an external service. Remmogo remains the owner of account, listing and rental data. The external service receives only the prepared report content needed to render the document and does not query the Remmogo database directly.

2. Operation summary

Item	Contract
Remmogo endpoint	POST /api/v1/admin/report-exports
Authentication	Bearer access token
Authorization	ADMINISTRATOR role required
Request content type	application/json
Success content type	application/json
Idempotency	Required Idempotency-Key header
Downstream dependency	PDF Rendering App POST /v1/documents
Expected outcome	A stored REPORT_EXPORT record and a downloadable PDF reference

3. Remmogo request

Headers

Authorization: Bearer <access-token>

Content-Type: application/json

Idempotency-Key: 3f90a462-9d1e-4e6b-bf7b-4bb4c861a4f1

Body

{

"reportType": "SEMESTER_RENTALS",

"periodStart": "2026-08-01",

"periodEnd": "2026-12-20",

"statusFilter": ["COMPLETED", "ACTIVE", "OVERDUE"],

"includeUserDetails": true

}

Field validation

Field	Required	Validation
reportType	Yes	Must be a supported value. Initial value: SEMESTER_RENTALS
periodStart	Yes	ISO 8601 date. Must not be after periodEnd.

periodEnd	Yes	ISO 8601 date. Must not be before periodStart or in an unapproved future reporting period.
statusFilter	No	Each value must be a valid RENTAL_STATUS; an omitted or empty list means all statuses.
includeUserDetails	No	Boolean; defaults to false. Detailed data remains restricted to Administrators.
Idempotency-Key	Yes	UUID or opaque value of 16-80 characters. The same key and body return the same export; the same key with a different body is rejected.

4. Processing rules

1. Authenticate the caller and verify the ADMINISTRATOR role.

2. Validate the request before creating an export record.

3. Reuse an existing result when the idempotency key and request fingerprint match.

4. Create REPORT_EXPORT with status PROCESSING in a transaction.

5. Query a consistent snapshot of qualifying rentals. The PDF app never receives database credentials.

6. Send the prepared title, period, generation timestamp, column definitions and report rows to the PDF

Rendering App.

7. If rendering succeeds, store the returned file reference and checksum, change the status to COMPLETED,

and return the result.

8. If the dependency times out or is unavailable, change the status to FAILED with a stable failure code. A

retry with the same idempotency key must not create a second export.

5. Downstream PDF Rendering App contract

Request

POST /v1/documents

Content-Type: application/json

X-Correlation-ID: 6d46b4c4-2e37-4bfb-a8bc-4b53af91afd8

{

"documentType": "table-report",

"outputFormat": "PDF",

"title": "Remmogo Semester Rental Report",

"fileName": "remmogo-rentals-2026-s1.pdf",

"metadata": {

"reportId": "8c964f13-0f07-420c-b45d-98aeaa086610",

"periodStart": "2026-08-01",

"periodEnd": "2026-12-20",

"generatedAt": "2026-10-12T17:00:00Z"

},

"columns": [

{"key": "rentalId", "label": "Rental ID"},

{"key": "itemName", "label": "Item"},

{"key": "customerName", "label": "Customer"},

{"key": "ownerName", "label": "Owner"},

{"key": "status", "label": "Status"},

{"key": "expectedReturnDate", "label": "Due Date"}

],

"rows": [

{

"rentalId": "R-1042",

"itemName": "Mini fridge",

"customerName": "Example Customer",

"ownerName": "Example Owner",

"status": "ACTIVE",

"expectedReturnDate": "2026-10-20"

}

]

}

Successful rendering response

HTTP/1.1 201 Created

Content-Type: application/json

{

"documentId": "pdf_01JAB6QTTQSCJGK3",

"status": "COMPLETED",

"downloadUrl": "https://pdf-app.example/documents/pdf_01JAB6QTTQSCJGK3",

"sha256": "d37c5f1c4f84c9c8f0b6b928c882c8d8074bde6a58f56c07166bf832431f3d3e",

"expiresAt": "2026-10-13T17:00:00Z"

}

6. Remmogo success response

HTTP/1.1 201 Created

Location: /api/v1/admin/report-exports/8c964f13-0f07-420c-b45d-98aeaa086610

Content-Type: application/json

{

"exportId": "8c964f13-0f07-420c-b45d-98aeaa086610",

"status": "COMPLETED",

"reportType": "SEMESTER_RENTALS",

"periodStart": "2026-08-01",

"periodEnd": "2026-12-20",

"recordCount": 527,

"downloadUrl": "/api/v1/admin/report-exports/8c964f13-0f07-420c-b45d-98aeaa086610/file",

"checksum": "d37c5f1c4f84c9c8f0b6b928c882c8d8074bde6a58f56c07166bf832431f3d3e",

"completedAt": "2026-10-12T17:00:06Z"

}

The system exposes its own authenticated download route rather than returning the external service URL to the browser.

7. Stable error outcomes

All errors use this structure:

{

"error": {

"code": "PDF_SERVICE_TIMEOUT",

"message": "The report could not be generated at this time.",

"correlationId": "6d46b4c4-2e37-4bfb-a8bc-4b53af91afd8",

"retryable": true

}

}

HTTP	Stable code Meaning	Client action
400	INVALID_REPORT_REQUEST	Dates, status values or required fields are invalid. Correct the request; do not retry unchanged.
401	AUTHENTICATION_REQUIRED	Token is absent, expired or invalid. Sign in again. HTTP Stable code Meaning Client action.
403	ADMIN_ROLE_REQUIRED	Authenticated caller is not an Administrator. Do not retry with the same account.
409	IDEMPOTENCY_KEY_REUSED	The key was previously used with a different body. Submit with a new key.
422	NO_REPORT_DATA	No rentals match the approved filters. Change the filter or period.
503	PDF_SERVICE_UNAVAILABLE	Rendering dependency is unavailable. Retry with the same idempotency key.
504	PDF_SERVICE_TIMEOUT	Rendering did not complete within the configured timeout. Retry with the same idempotency key.
500	REPORT_EXPORT_FAILED	An unexpected internal failure occurred. Contact support with the correlation ID.

8. Security and privacy

• Only Administrators may generate or download administrative reports.

• Remmogo sends only fields required for the selected report.

• Access tokens and password hashes are never included in rendering payloads.

• External download URLs are not exposed directly to clients.

• Transport uses TLS, service credentials are stored outside source code, and logs exclude report-row personal data.

• Generated files have an expiry/retention policy and an audit record identifies the requesting

Administrator.

9. Failure and recovery behaviour

The principal failure risk is a timeout after the PDF app has accepted a request but before Remmogo receives the response. Without idempotency, retrying could generate duplicate files. Remmogo therefore stores the client idempotency key and a request fingerprint before calling the dependency, propagates a correlation identifier, and returns the existing export on a matching retry. A background reconciliation operation may query the PDF app by correlation identifier before starting another render.

Verification test: Simulate a timeout after the PDF app stores the document. Retry the identical request with the same idempotency key. Confirm that Remmogo returns one REPORT_EXPORT, one external document and the same exportId; no duplicate report is created.

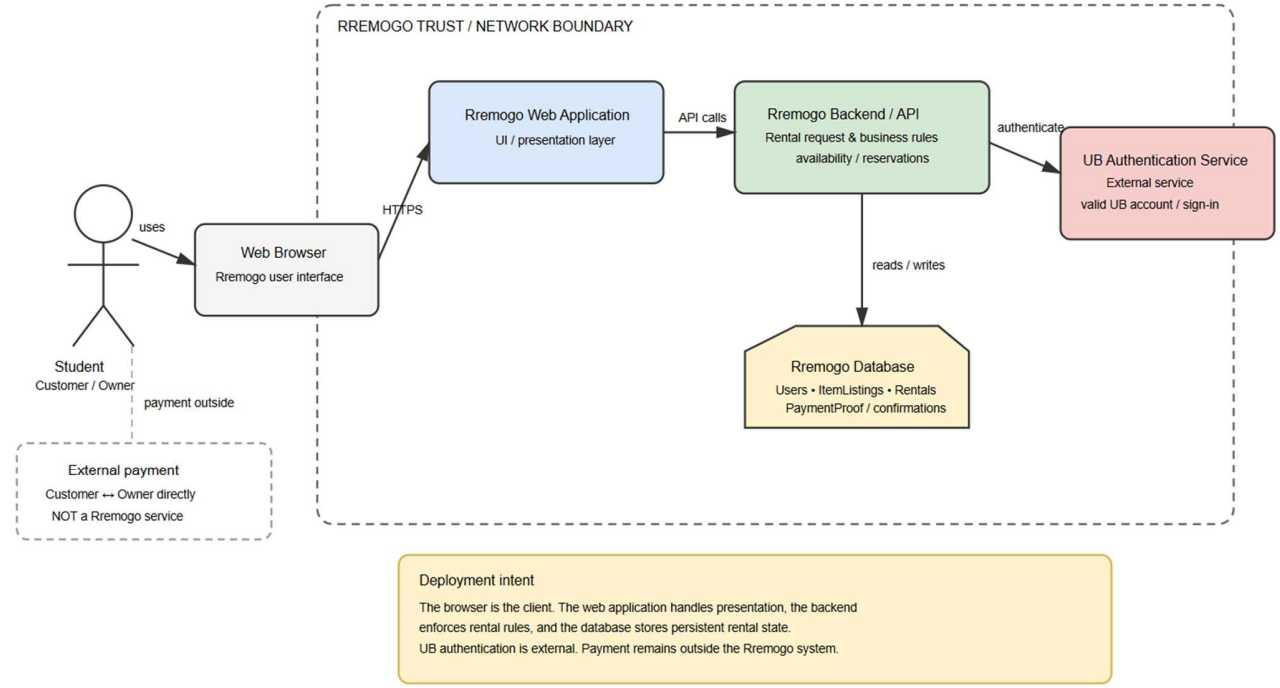
10. Data impact

The operation reads RENTAL, ITEM_LISTING, STUDENT and related account data from a consistent snapshot. It writes one REPORT_EXPORT record. It does not modify rental status, item availability, payment confirmations or return confirmations.

Deployment

The system will be deployed using a simple three-tier architecture:

Rremogo — Deployment Diagram



Failure and Recovery

Database Failure

- System displays an error message.
- No transaction is saved until database connection is restored.

Payment Confirmation Failure

- Rental remains pending.
- System waits for confirmation until timeout.

Chat Service Failure

- Messages are stored and delivered when service reconnects.

Application Crash

- Data remains safe in the database.
- Users can continue once the application restarts.

Duplicate booking Scenario: Two customers Request the same item

Normal Scenario

1.Customer submits rental request
2.System checks quantity
3.Quantity = 1
4.System reserves item
5.Rental request accepted

Failure Scenario

1.Customer A requests laptop
2.Customer B requests same laptop
3.Both requests arrive simultaneously

Recovery Mechanism

1.Database transaction begins
2.System locks listing record
3.Availability checked
4.First request reserves item
5.Quantity updated
6.System unlocks listing record
7.Second request rechecks quantity
8.System returns
9.ITEM_NOT_AVAILABLE

Outcome:

1.Only one request succeeds.
2.Data remains consistent.

11. Detailed design, patterns and framework

Interfaces

Dependency direction

```
Web controllers / scheduled jobs
        |
        v
    Inbound ports
RentalUseCase, ReportExportUseCase
        |
        v
 Application services and domain rules
        |
        v
    Outbound ports
RentalRepository, ItemAvailabilityPort,
NotificationPort, PdfReportPort
        ^
        |
Database, email and PDF-service adapters
```

Dependencies point towards the application core. Controllers call inbound ports. Application services depend on outbound-port interfaces. Concrete database, notification and PDF clients implement those outbound interfaces.

Framework annotations such as Spring @RestController, JPA @Entity and HTTP-client configuration belong in adapters, not in the port interfaces.

Port catalogue

Port	Direction	Called by	Implemented by	Responsibility
RentalUseCase	Inbound / provided	Student and Owner controllers	RentalApplicationService	Request a rental, accept/reject it and record payment or return confirmations.
ReportExportUseCase	Inbound / provided	Administrator report controller	ReportExportService	Start or retrieve an idempotent PDF report export.
RentalRepository	Outbound / required	Rental application service	Database repository adapter	Load and save rental aggregates while protecting concurrent updates.
ItemAvailabilityPort	Outbound / required	Rental application service	Listing/JPA adapter	Atomically reserve or release item quantity.
NotificationPort	Outbound / required	Rental and overdue services	Email/in-app notification adapter	Send secondary notifications after the core state change succeeds.
PdfReportPort	Outbound / required	Report export service	HTTP adapter for the PDF application	Render structured report content as a PDF without exposing the Remmogo database.

Inbound ports

RentalUseCase

RentalUseCase is the application boundary for the core rental workflow. It accepts commands containing identifiers and user-entered values, applies authorisation and business rules, coordinates availability and persistence, and returns a stable result.

```
public interface RentalUseCase {
  RentalResult requestRental(RequestRentalCommand command);
  RentalResult decideRental(DecideRentalCommand command);
  RentalResult confirmPayment(ConfirmRentalEventCommand command);
  RentalResult confirmReturn(ConfirmReturnCommand command);
}
```

The controller is responsible for HTTP parsing and basic format validation. The application service remains responsible for rules that require current domain data, including:

- the Customer and Owner must be different Students;
- the listing must be active and have enough available quantity;
- only the listing Owner may accept or reject the request;
- only rental participants may confirm payment or return;
- each party may confirm each event once;
- invalid lifecycle transitions are rejected.

ReportExportUseCase

```
public interface ReportExportUseCase {
  ReportExportResult generateReport(GenerateReportCommand command);
  Optional<ReportExportResult> findExport(UUID exportId, UUID administratorId);
}
```

The inbound port requires an idempotency key. Repeating the same request with the same key returns the existing export; reusing the key with different filters is rejected.

Outbound ports

RentalRepository

```
public interface RentalRepository {
  Optional<RentalSnapshot> findById(UUID rentalId);
  Optional<RentalSnapshot> findByIdempotencyKey(String key);
  RentalSnapshot save(RentalSnapshot rental);
}
```

The interface uses application/domain models rather than JPA entities. The adapter translates between RentalSnapshot and the persistence representation.

ItemAvailabilityPort

```
public interface ItemAvailabilityPort {
  AvailabilityResult reserve(UUID listingId, int quantity, long expectedVersion);
  AvailabilityResult release(UUID listingId, int quantity, long expectedVersion);
}
```

This port represents an atomic business operation, not generic CRUD. reserve succeeds only when the listing is active and sufficient quantity remains. The expected version allows optimistic concurrency control. Two Customers requesting the final unit must produce exactly one success.

NotificationPort

```
public interface NotificationPort {
  void rentalRequested(RentalNotification notification);
  void rentalStatusChanged(RentalNotification notification);
  void rentalOverdue(RentalNotification notification);
}
```

Notifications are secondary effects. The rental transaction is committed first, and notification failure must not roll back or duplicate the rental. Failed deliveries are logged for retry using the rental ID and event type as an idempotency reference.

PdfReportPort

```
public interface PdfReportPort {
  PdfRenderResult render(PdfRenderRequest request);
  Optional<PdfRenderResult> findByCorrelationId(String correlationId);
}
```

The PDF adapter converts the application request into the external application's HTTP/JSON format. The core service never depends on the external client's SDK or response classes.

The adapter must:

- use TLS and service credentials stored outside source code;
- apply connection and response timeouts;
- translate external failures into stable application outcomes;
- propagate the correlation identifier;
- avoid logging report rows containing personal data;
- reconcile an unknown timeout outcome before starting another render.

6 Application-service coordination

Request rental

1. RentalController maps the authenticated Student and JSON body to RequestRentalCommand.
2. RentalApplicationService, implementing RentalUseCase, checks for an existing idempotent result.
3. It loads the listing and verifies the Student is not its Owner.
4. It calls ItemAvailabilityPort.reserve(...).
5. If reservation succeeds, it creates and stores the REQUESTED rental through RentalRepository in the same transaction boundary.
6. After commit, it calls NotificationPort.rentalRequested(...).
7. The controller maps the result to the API response.

If reservation fails, no rental is stored and the application returns ITEM_UNAVAILABLE. If the persistence transaction fails after reservation, the transaction must roll back both changes or execute a compensating release before returning failure.

Generate PDF report

1. AdminReportController maps filters and the Idempotency-Key header to GenerateReportCommand.
2. ReportExportService, implementing ReportExportUseCase, authorises the Administrator and checks for an existing export.
3. It queries a consistent report-data snapshot and stores the export as PROCESSING.
4. It calls PdfReportPort.render(...) with prepared columns and rows.
5. On success, it stores the file reference, checksum and COMPLETED status.
6. On a known failure, it stores FAILED with a stable code.
7. On timeout, it preserves the correlation ID and checks findByCorrelationId(...) before allowing a new render.

Stable outcomes and exceptions

Port interfaces return explicit results for expected business outcomes. Exceptions are reserved for unexpected technical failures.

Outcome	Meaning	API mapping
SUCCESS	Operation completed.	200 or 201
ITEM_UNAVAILABLE	Listing has insufficient quantity or is inactive.	409
INVALID_STATE	Requested lifecycle transition is not permitted.	409
FORBIDDEN	Authenticated user is not allowed to perform the action.	403
NOT_FOUND	Requested rental, listing or export does not exist.	404
DUPLICATE_CONFIRMATION	The same party already confirmed the event.	409; return current state
PDF_UNAVAILABLE	PDF dependency is unavailable.	503
PDF_TIMEOUT	PDF outcome is unknown and requires reconciliation.	504 or pending response

Dependency injection

Application services receive required ports through constructor injection:

```
public final class RentalApplicationService implements RentalUseCase {
  private final RentalRepository rentals;
  private final ItemAvailabilityPort availability;
  private final NotificationPort notifications;

  public RentalApplicationService(
      RentalRepository rentals,
      ItemAvailabilityPort availability,
      NotificationPort notifications) {
    this.rentals = rentals;
    this.availability = availability;
    this.notifications = notifications;
  }
}
```

Constructor injection makes dependencies explicit, supports unit tests with in-memory fakes and prevents the core service from locating dependencies through global state or a service locator.

Testing the boundaries

Test	Ports replaced	Expected evidence
Two concurrent requests for the last unit	Fake/transactional ItemAvailabilityPort	One succeeds, one returns ITEM_UNAVAILABLE, quantity never becomes negative.
Duplicate rental submission	In-memory RentalRepository	Same idempotency key returns the same rental ID.
Duplicate payment confirmation	In-memory repository	One confirmation per party; repeat returns current state without adding another row.
Notification outage	Failing NotificationPort	Rental remains stored once and notification is marked for retry.
PDF timeout after external acceptance	Fake PdfReportPort	Reconciliation returns the original document; no duplicate export is produced.
PDF service unavailable	Failing PdfReportPort	Export receives stable failure code; rental data remains unchanged.

Cost and benefit

The greatest benefit comes from ItemAvailabilityPort: it makes the double-booking rule explicit and independently testable while allowing the database adapter to implement locking or optimistic concurrency. The cost is additional interfaces, mapping code and more classes.

The design will be confirmed by a concurrency integration test in which two requests compete for the final available unit. The architecture is accepted only if exactly one reservation succeeds and the stored quantity never becomes negative.

Not every class receives an interface. Simple value objects, domain records and framework-independent utilities remain concrete because an interface would add indirection without

providing a meaningful boundary.

Patterns

Repository Pattern

The Repository Pattern separates database access from the application's business logic. Instead of allowing controllers or services to communicate directly with the database, the application uses repository classes to handle data retrieval, storage, updating and deletion. This keeps database-related code in one place and makes the application easier to maintain and test.

The repositories identified for Remmogo are:

- UserRepository – handles data operations relating to user accounts and user information.

ItemRepository – handles data operations for item listings, including retrieving and updating listing information.
RentalRepository – handles data operations relating to rental records and their status.
RatingRepository – handles data operations relating to ratings and reviews.

Using repositories gives the application a clearer separation between data access and business rules. It also makes it easier to test application logic without placing database operations directly inside controllers.

Service Layer Pattern

The Service Layer Pattern places the application's business rules in service classes between the controllers and repositories. Controllers receive requests and coordinate the response, while services carry out the relevant operations and apply the rules required by the system. Services use repositories when they need to read or update stored data.

The services identified for Remmogo are:

UserService – contains user-related operations and business rules, such as handling user account activities.
RentalService – manages rental-related operations and applies the rules governing rental requests and their processing.
ItemService – manages item-related operations and applies the rules for creating, updating and managing item listings.

This pattern helps keep controllers focused on handling requests rather than containing large amounts of business logic. It also supports reuse, since the same service operations can be called from different parts of the application when required.

Model–View–Controller (MVC) Pattern

The Model–View–Controller (MVC) Pattern separates the user interface and request-handling responsibilities into three parts:

Model – represents the application's data and domain objects, including User, Item, Rental and Rating.
View – represents the pages and interface elements presented to users, implemented through the application's HTML and CSS interfaces.
Controller – handles incoming requests, coordinates the required application operations and returns an appropriate response. The controllers identified for Remmogo are UserController, ItemController, RentalController and AdminController.

MVC provides an organised way to structure the presentation and request-handling parts of the application. It complements the Three-Tier Architecture used by Remmogo; it does not replace that architecture. The Three-Tier Architecture separates the presentation, business-logic and data-access responsibilities, while MVC organises related responsibilities within the application.

How the Patterns Work Together

The three patterns support different responsibilities within Remmogo. Controllers handle incoming requests and coordinate responses. They call the appropriate service, which applies the system's business rules. When data must be accessed or changed, the service uses the relevant repository. The repository performs the data operation and returns the result to the service. This arrangement keeps request handling, business logic and database access separate, making the system easier to understand, test and maintain.

Rejected alternatives

Direct Database Access

Business services execute SQL queries directly.

Rejected because:

- Creates tight coupling between business logic and the database.
- Difficult to test.
- Harder to maintain.

Framework Boundary

Frontend

- HTML
- CSS
- JavaScript

Backend

- Node.js
- Express.js

Database

- MySQL

Reasons for the Decision

- Supports the Three-Tier Architecture selected for the project.
- Integrates well with MySQL databases.
- Has a large developer community and documentation.

- Node.js and Express.js are lightweight, and easy to learn.

The project will use npm (Node Package Manager) to manage external libraries and project dependencies.

Examples of dependencies include:

- Node.js
- Express.js
- MySQL2
- jsonwebtoken

Positive Consequences

- Faster development.
- Easy installation of project libraries.
- Large amount of community support.
- Good integration with web technologies.

Negative Consequences

- Dependencies must be updated regularly.
- Some third-party packages may introduce security vulnerabilities if not maintained.

12. User-interface, accessibility and usability

Task flow

Request to Rent an Item

A UB student needs an item for a limited period. The student signs in to Remmogo, searches for the item and checks its details. If the item is available, the student selects the required rental period and submits a request.

The system checks availability and records the request. The customer should receive a clear confirmation of the request status and information about the next steps.

Preconditions

- The student has a valid UB account.
- The student is signed in to Remmogo.
- The item has been listed on the platform.
- The item is available for the requested period.

Main Success Flow

Step	User Action	System Response
1	The student opens Remmogo.	The system displays the available marketplace functions
2	The student searches for an item or browses the listings	The system displays matching items and their availability.
3	The student selects an item	The system displays the item's description, condition, rental price and availability
4	The student selects the required rental period	The system accepts the selected period for validation
5	The student submits the rental request	The system checks whether the item is available for that period
6	The request passes validation	The system records the request and updates the relevant reservation status.
7	The student views the confirmation	The system displays the request status and explains the next steps

Postconditions

If the request is accepted, the system records the rental request and reflects the appropriate reservation status. The customer can view the status and follow the next steps in the rental process.

A successful request does not necessarily mean that the rental is fully active. Payment confirmation and handover may still need to be completed according to the agreed rental process.

3. Alternative and Error Scenarios

Situation	System Response

The item is unavailable.	The system informs the customer that the item cannot be requested for the selected period.
The customer does not select a rental period	The system identifies the missing information and asks the customer to provide it.
The submitted information is invalid	The system explains what needs to be corrected.
The request is still processing	The system provides feedback that the request is being processed and prevents unnecessary repeated submissions.
A system error occurs.	The system displays an understandable error message and advises the customer to try again.
The customer's session expires.	The system asks the customer to sign in again before continuing
The customer's session expires.	The system asks the customer to sign in again before continuing.
The item becomes unavailable before the request is confirmed.	The system rejects the request and informs the customer that the item's availability has changed.

List an Item for Rent

A UB student owns an item that can be rented by other students. The owner signs in to Remmogo and creates a new listing by entering the item details. The owner provides information such as the item name, category, description, condition, rental price and available quantity.

The system validates the information and publishes the listing so that other users can find it through the system.

Preconditions

- The owner has a valid UB account.
- The owner is signed in to Remmogo.
- The item belongs to the owner.
- The owner has permission to create listings.

Main Success Flow

Step	User Action	System Response

1	The owner opens Remmogo.	The system displays the available marketplace functions.
2	The owner selects the option to create a new listing.	The system displays the item listing form.
3	The owner enters the item details.	The system accepts the information for validation.
4	The owner selects an item category.	The system records the selected category.
5	The owner enters the rental price and available quantity.	The system validates the entered values.
6	The owner submits the listing.	The system validates all required information.
7	The listing passes validation.	The system creates the item listing and stores it in the database.
8	The owner views the result.	The system displays a confirmation that the item has been listed successfully.
9	The owner returns to the marketplace.	The system displays the newly listed item as available for rent.

Postconditions

If the listing is accepted, the item is stored in the system and becomes visible to other users searching the marketplace.

The item is available for rental requests according to the quantity specified by the owner.

Alternative and Error Scenarios

Situation	System Response
Required information is missing.	The system highlights the missing fields and asks the owner to complete them.
The owner does not select a category.	The system prevents submission and requests a valid category.
The rental price is invalid.	The system explains the error and asks the owner to enter a valid rental price.
The available quantity is invalid.	The system requests a valid quantity greater than zero.

The listing contains unsupported or invalid information.	The system identifies the information that must be corrected.
The owner attempts to submit the form multiple times.	The system prevents duplicate submissions and displays the current submission status.
A system error occurs.	The system displays an understandable error message and advises the owner to try again.
The owner's session expires.	The system asks the owner to sign in again before continuing.

Return a Rented Item

A customer has finished using a rented item and wants to return it to its owner. The customer confirms the return in Remmogo after handing the item back. The owner then verifies that the item has been returned.

The system records both confirmations and completes the rental transaction. The item becomes available for future rentals and both parties may leave ratings.

Preconditions

- The customer has a valid UB account.
- The owner has a valid UB account.
- The rental transaction exists.
- The item is currently in an active rental state.
- The customer has returned the item to the owner.

Main Success Flow

Step	User Action	System Response
1	The customer signs in to Remmogo.	The system displays the customer's active rentals.
2	The customer selects the rental record.	The system displays the rental details and return option.
3	The customer confirms that the item has been returned.	The system records the customer's return confirmation.
4	The owner signs in to Remmogo.	The system displays rentals awaiting return confirmation.

5	The owner opens the rental record.	The system displays the rental and return details.
6	The owner confirms that the item has been received.	The system records the owner's return confirmation.
7	Both confirmations have been received.	The system marks the rental as completed.
8	The transaction is completed.	The system updates the item's availability and quantity.
9	The customer and owner view the completed transaction.	The system allows both parties to submit ratings and reviews.

Postconditions

If the return process is completed successfully, the rental transaction is marked as completed and the item's availability is updated.

Both the customer and owner may rate each other based on the completed rental transaction.

Alternative and Error Scenarios

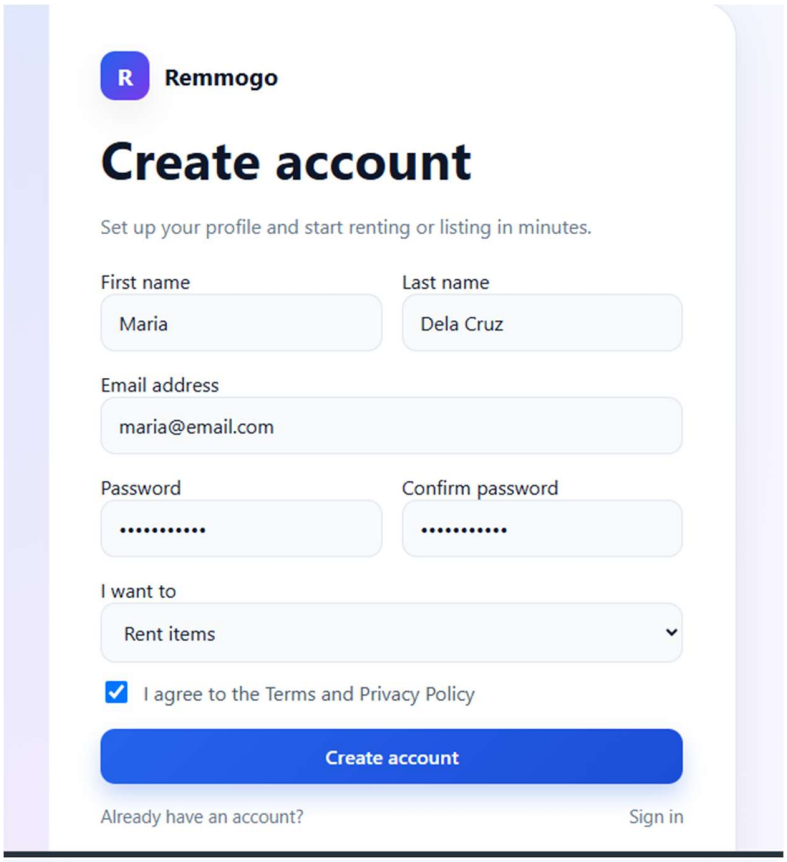
Situation	System Response
The customer attempts to confirm a return for a rental that does not exist.	The system rejects the request and displays an error message.
The owner has not yet confirmed the return.	The system records the customer's confirmation and waits for the owner's confirmation.
The customer has not yet confirmed the return.	The system records the owner's confirmation and waits for the customer's confirmation.
The item is reported as damaged or incomplete.	The system records the issue and flags the rental for administrator review.
The rental is already marked as completed.	The system informs the user that no further return confirmation is required.
The user submits the confirmation multiple times.	The system prevents duplicate confirmations and displays the current status.

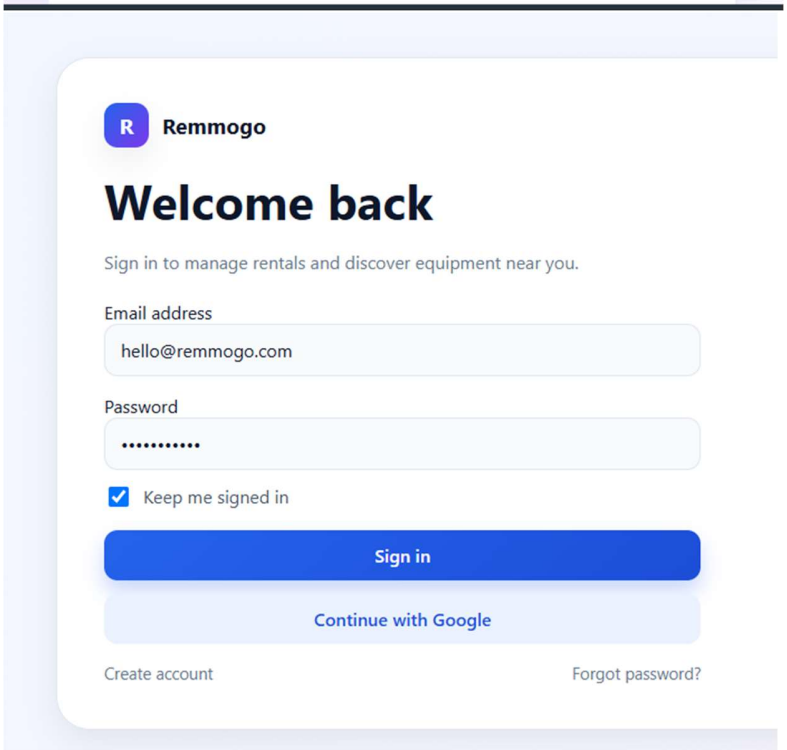
A system error occurs.	The system displays an understandable error message and advises the user to try again.
The user's session expires.	The system asks the user to sign in again before continuing.

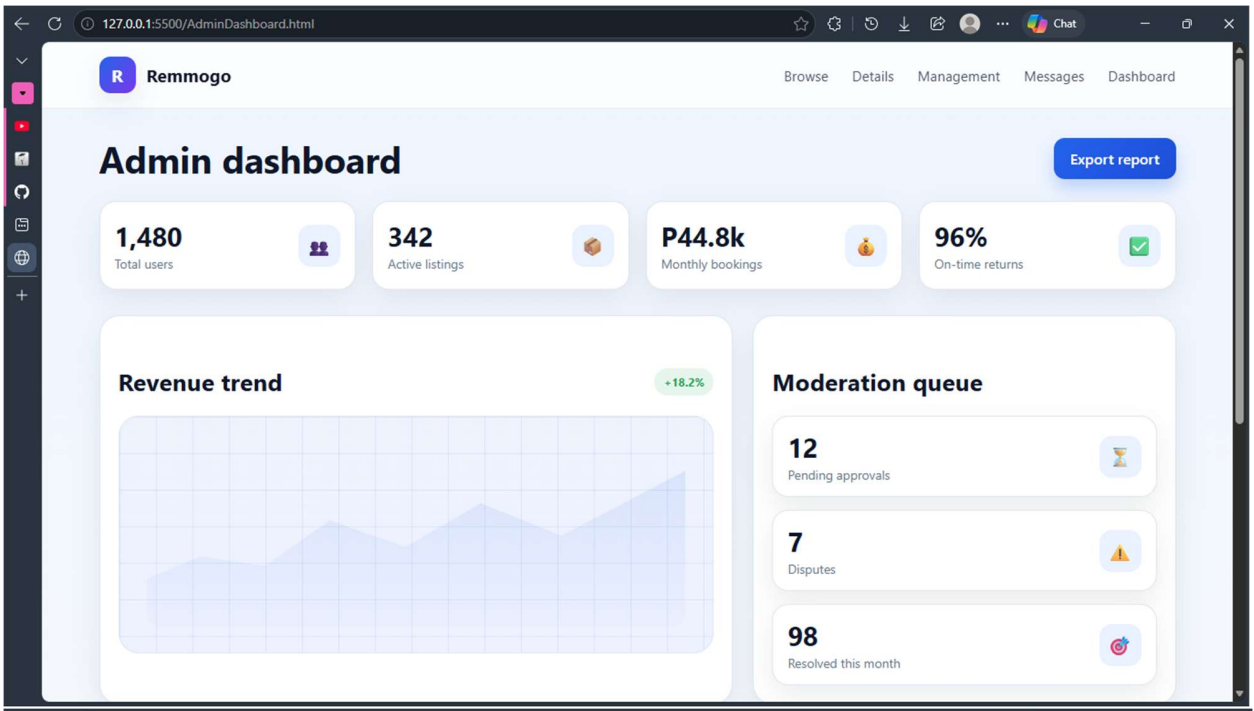
Wireframes

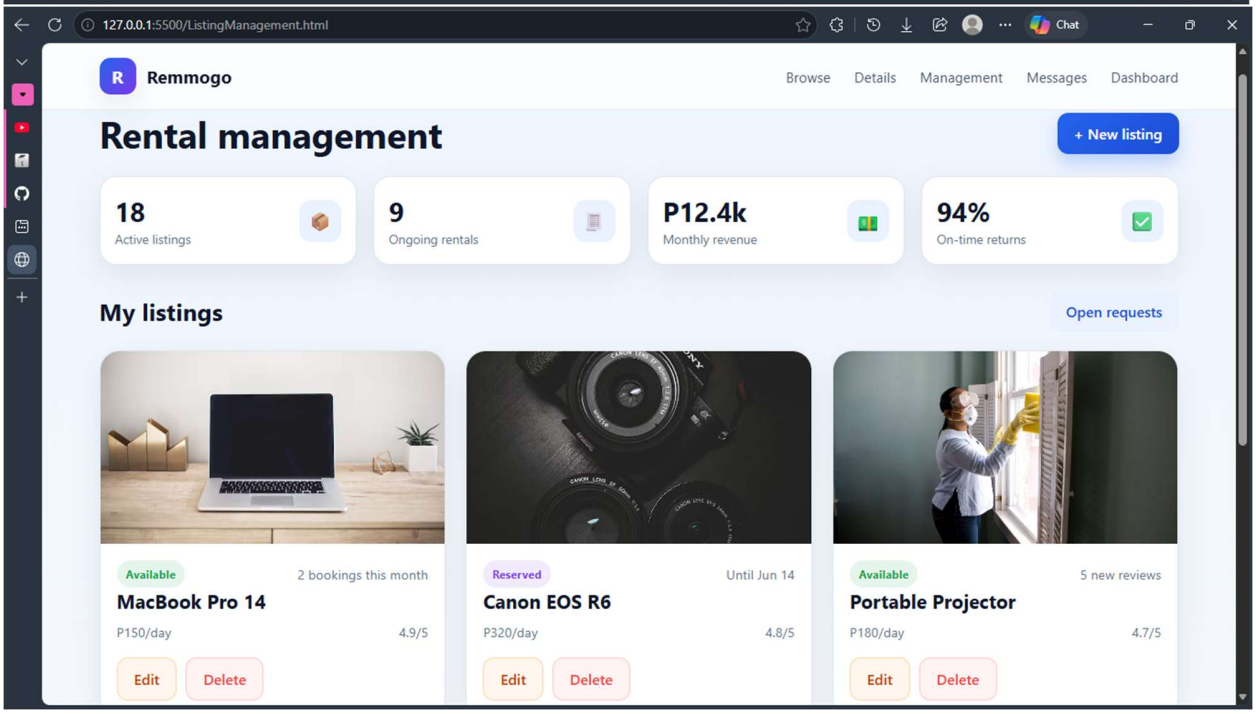
Main wireframes developed:

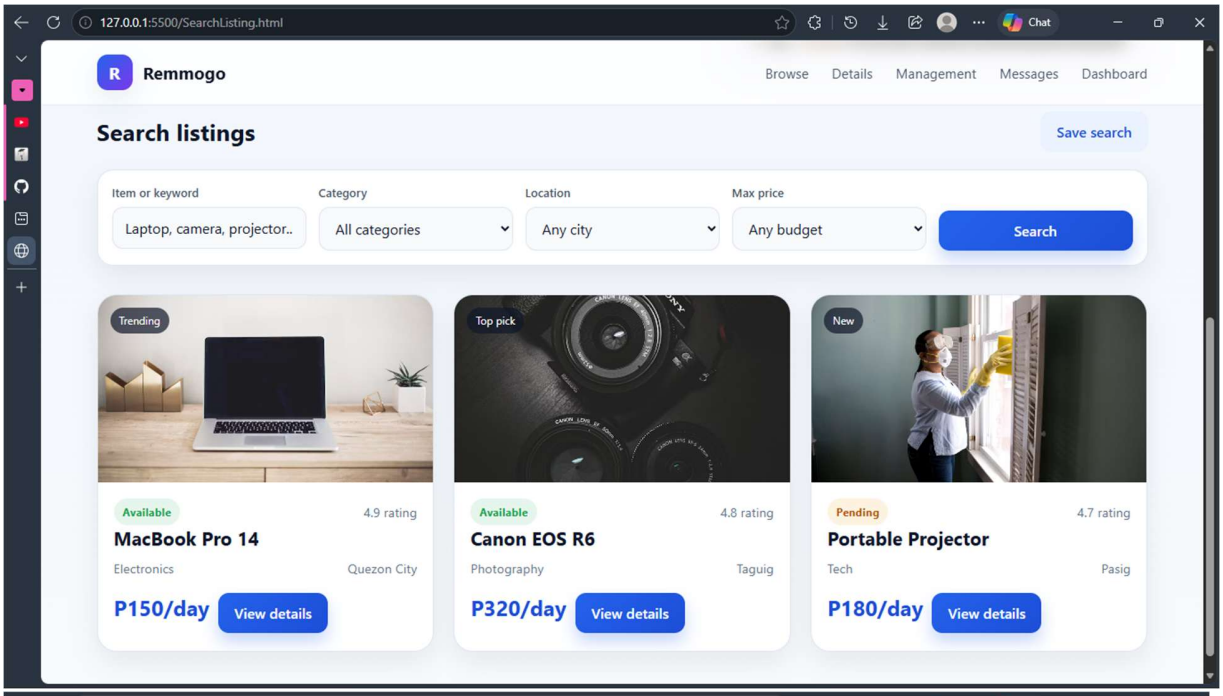
- Login Page
- Registration Page
- Item Search Page
- Item Details Page
- Rental Management Page
- Chat Page
- Admin Dashboard

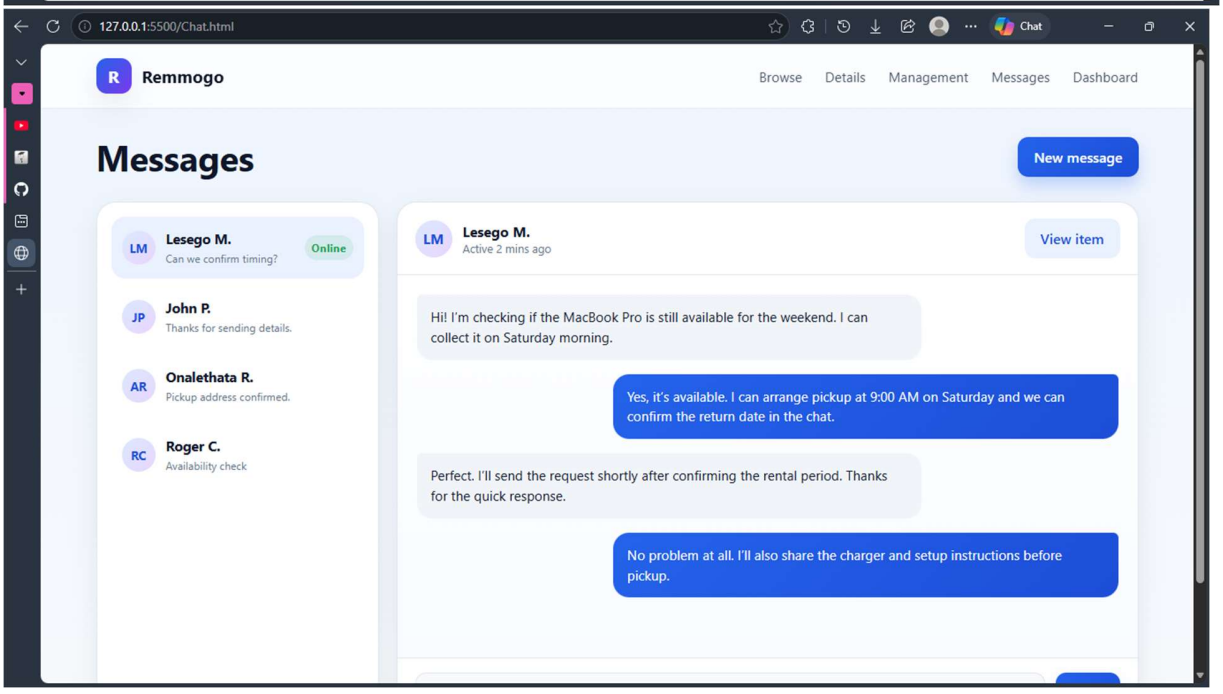

R
Remmogo
Create account
Set up your profile and start renting or listing in minutes.
First name
Maria
Last name
Dela Cruz
Email address
maria@email.com
Password
Confirm password
I want to
Rent items
I agree to the Terms and Privacy Policy
Create account
Already have an account?
Sign in

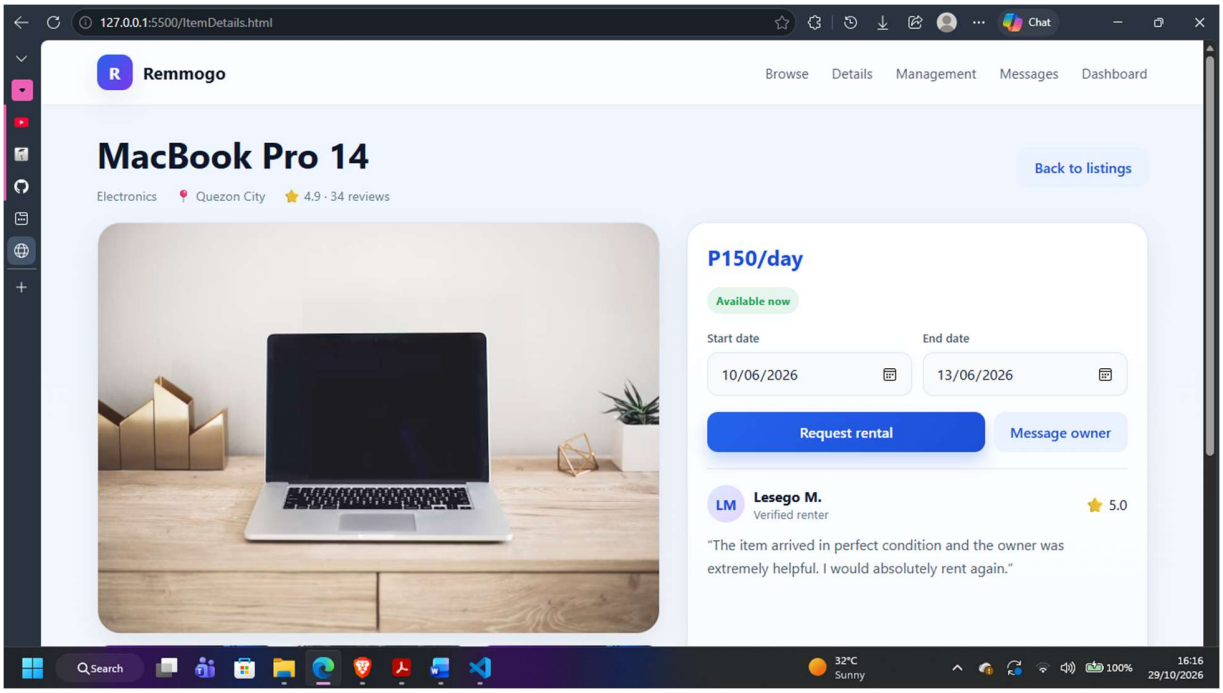

R
Remmogo
Welcome back
Sign in to manage rentals and discover equipment near you.
Email address
hello@remmogo.com
Password
Keep me signed in
Sign in
Continue with Google
Create account
Forgot password?

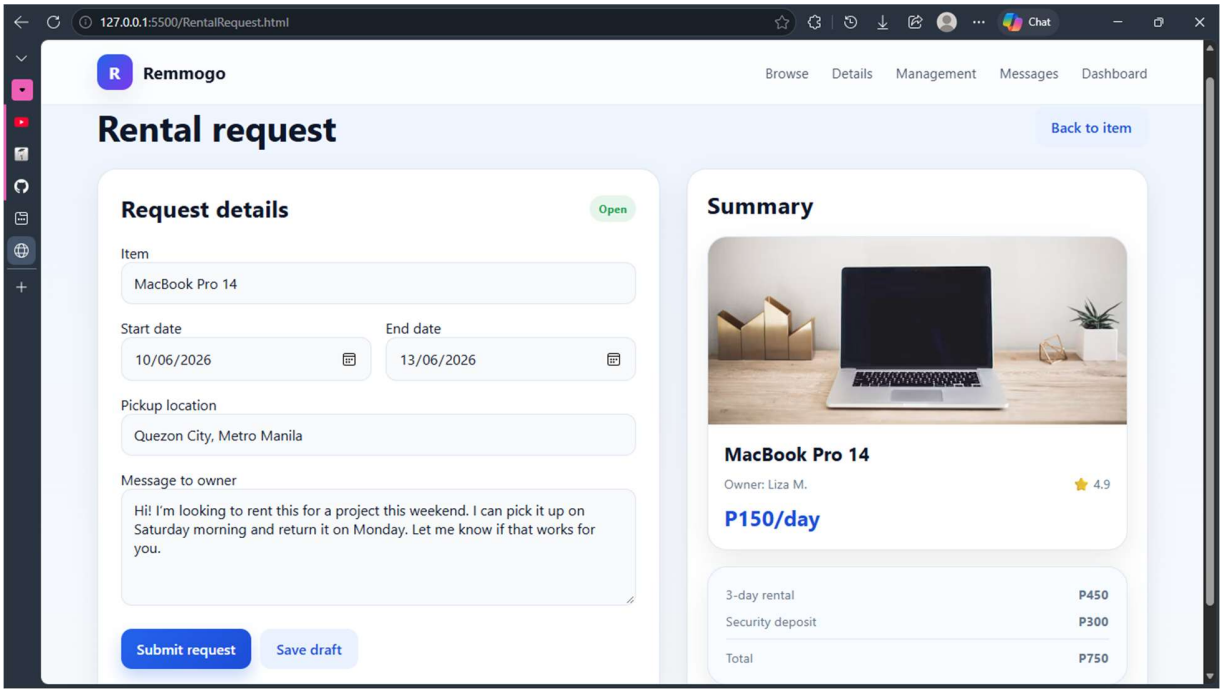

127.0.0.1:5500/AdminDashboard.html
Chat
R
Remmogo
Browse
Details
Management
Messages
Dashboard
Admin dashboard
Export report
1,480
Total users
342
Active listings
P44.8k
Monthly bookings
96%
On-time returns
Revenue trend
+18.2%
Moderation queue
12
Pending approvals
7
Disputes
98
Resolved this month


127.0.0.1:5500/ListingManagement.html
Chat
R
Remmogo
Browse
Details
Management
Messages
Dashboard
Rental management
+ New listing
18
Active listings
9
Ongoing rentals
P12.4k
Monthly revenue
94%
On-time returns
My listings
Open requests
Available
2 bookings this month
MacBook Pro 14
P150/day
4.9/5
Edit
Delete
Reserved
Until Jun 14
Canon EOS R6
P320/day
4.8/5
Edit
Delete
Available
5 new reviews
Portable Projector
P180/day
4.7/5
Edit
Delete


127.0.0.1:5500/SearchListing.html
Chat
Remmogo
Browse
Details
Management
Messages
Dashboard
Search listings
Save search
Item or keyword
Laptop, camera, projector..
Category
All categories
Location
Any city
Max price
Any budget
Search
Trending
Top pick
New
Available
4.9 rating
MacBook Pro 14
Electronics
Quezon City
P150/day
View details
Available
4.8 rating
Canon EOS R6
Photography
Taguig
P320/day
View details
Pending
4.7 rating
Portable Projector
Tech
Pasig
P180/day
View details


127.0.0.1:5500/Chat.html
Chat
Remmogo
Browse
Details
Management
Messages
Dashboard
Messages
New message
LM
Lesego M.
Can we confirm timing?
Online
JP
John P.
Thanks for sending details.
AR
Onalethata R.
Pickup address confirmed.
RC
Roger C.
Availability check
LM
Lesego M.
Active 2 mins ago
View item
Hi! I'm checking if the MacBook Pro is still available for the weekend. I can collect it on Saturday morning.
Yes, it's available. I can arrange pickup at 9:00 AM on Saturday and we can confirm the return date in the chat.
Perfect. I'll send the request shortly after confirming the rental period. Thanks for the quick response.
No problem at all. I'll also share the charger and setup instructions before pickup.


127.0.0.1:5500/ItemDetails.html
Chat
Remmogo
Browse
Details
Management
Messages
Dashboard
MacBook Pro 14
Back to listings
Electronics
Quezon City
4.9 · 34 reviews
P150/day
Available now
Start date
End date
10/06/2026
13/06/2026
Request rental
Message owner
LM
Lesego M.
Verified renter
5.0
"The item arrived in perfect condition and the owner was extremely helpful. I would absolutely rent again."
Search
32°C
Sunny
100%
16:16
29/10/2026


127.0.0.1:5500/RentalRequest.html
Chat
Remmogo
Browse
Details
Management
Messages
Dashboard
Rental request
Back to item
Request details
Open
Item
MacBook Pro 14
Start date
End date
10/06/2026
13/06/2026
Pickup location
Quezon City, Metro Manila
Message to owner
Hi! I'm looking to rent this for a project this weekend. I can pick it up on Saturday morning and return it on Monday. Let me know if that works for you.
Submit request
Save draft
Summary
MacBook Pro 14
Owner: Liza M.
4.9
P150/day
3-day rental
P450
Security deposit
P300
Total
P750

Alternative States

Search and available-items states

State	Trigger	Interface treatment	Available actions	Accessibility and integrity notes
Initial	Customer opens the storefront with no query.	Show search field, category filter and currently available items. Search field has a persistent label: **Search by item name**.	Search, filter, open listing.	Results region has a heading. Do not use placeholder text as the only label.
Loading	Search query or filter changes.	Keep the previous results visible but visually mark them as updating. Show **Searching available items...** and skeleton rows/cards.	Cancel or change query; prevent repeated identical requests.	Loading text uses aria-live="polite"; skeletons are hidden from assistive technology.
Results	Matching available listings are returned.	Show item name, category, condition, price, available quantity and **View details** action.	Sort, filter, open item.	Result count is announced. Cards follow a logical keyboard order.
Empty query result	Search succeeds	Show **No available items match your**	Clear filters; search again; browse all	This is not presented as an error.

State	Trigger	Interface treatment	Available actions	Accessibility and integrity notes
	but nothing matches.	**search**. Retain query and filters.	available items.	
Empty storefront	No listings are currently available.	Show **No items are available right now** with a brief explanation that reserved and unavailable items are hidden.	Refresh; return later.	Do not display unavailable items as if they can be requested.
Search error	Search service fails.	Show **We could not load available items** and confirm that no action was submitted. Keep the query.	Try again; clear filters; return to dashboard.	Move focus to the error summary only after a submitted search; retain the visible field label.

Item-details and rental-request states

State	Trigger	Interface treatment	Available actions	Accessibility and integrity notes
Available	Listing is active with availableQuantity > 0.	Show item details, unit price, available quantity, Owner, date fields, quantity field and **Send**	Enter dates and quantity; send request; chat with Owner.	Price and quantity use text labels, not icons alone.

State	Trigger	Interface treatment	Available actions	Accessibility and integrity notes
		rental request.		
Loading details	Customer opens an item.	Show **Loading item details…** and a stable page structure.	Back to results.	Page title updates when content arrives.
Request accepted	Rental is stored successfully.	Show **Rental request sent** with rental reference, item, dates, quantity and status REQUESTED.	View rental; message Owner; return to storefront.	Announce success using a live region; do not rely on a green colour alone.
Listing unavailable	Owner removes/deactivates the listing or available quantity reaches zero.	Replace request form with **This item is currently unavailable**.	Return to results; refresh.	Request action is removed, not merely greyed out without explanation.
Stale details	Item data changes while the form is open.	Show **Item details changed since you opened this page** and	Review changes; accept and continue; cancel.	Require explicit review if the agreed price changed.

State	Trigger	Interface treatment	Available actions	Accessibility and integrity notes
		highlight changed price, quantity or status using text.		
Submission timeout	No response arrives within the configured period.	Show **We could not confirm whether your request was received**. Do not invite the user to submit a new request immediately.	Check My Rentals; retry status check.	Reconcile using the request/idempotency key so retry cannot create a duplicate rental.
General submission error	Request is rejected for a recoverable internal error.	Show **Rental request could not be sent** and retain the form.	Try again; contact support if repeated.	State that no confirmed request exists unless

Owner task states

Task

The Owner reviews an incoming rental request, confirms payment and handover, and later confirms the returned item.

State	Trigger	Interface treatment	Available actions	Integrity notes
No pending requests	Owner has no requests requiring action.	Show **No rental requests need your attention**.	View all rentals; manage listings.	Empty state is not an error.
Request awaiting decision	Rental status is REQUESTED.	Show Customer, item, quantity, dates and request time.	Accept; reject; message Customer.	Recheck availability in the acceptance transaction.
Request already changed	Customer cancelled, another action completed or stale screen submitted.	Show the current status and **This request has already changed**.	Refresh; open current rental.	Do not overwrite a newer state.
Waiting for Customer payment confirmation	Owner has not yet received the Customer's confirmation.	Show **Waiting for Customer confirmation** and the rental status.	Message Customer.	Remmogo records confirmation only; it does not process money.
Owner confirmation required	Customer has confirmed payment.	Show **Confirm payment received** with rental reference and amount.	Confirm; message Customer.	Confirmation action must identify the confirming party.
Confirmation saved	Owner confirmation succeeds.	Show both parties' confirmation states and the next lifecycle action.	Confirm handover when permitted.	Only one confirmation per party per rental.

State	Trigger	Interface treatment	Available actions	Integrity notes
Return confirmation pending	Only one party has confirmed the return.	Show who has confirmed and **Waiting for [Customer/Owner]**.	Confirm return if eligible; message other party.	Item quantity is restored only according to the agreed lifecycle rule, not on repeated clicks.
Overdue	Due date passes without completed return confirmation.	Show **Rental overdue** with due date and required action.	Message party; confirm return; report issue.	Status text and warning icon are used together; overdue is not colour-only.

Administrator PDF-report states

Task

The Administrator selects a reporting period and rental statuses, generates a semester rental report and downloads the completed PDF.

Success criteria

- Only an Administrator can generate or download the report.
- Validation prevents invalid reporting periods.
- One request creates at most one REPORT_EXPORT record and one external PDF document.
- A report containing more than 500 records is normally ready within ten seconds under QS-05.
- Service failures give stable messages and safe retry behaviour.

State	Trigger	Interface treatment	Available actions	API/error mapping and accessibility notes
Initial	Administrator opens Generate Report.	Show report type, labelled start/end dates, status filters and **Generate PDF**.	Set filters; generate.	Explain that the report may contain personal information and is for administrative use.
Validation error	Missing/invalid dates or unsupported filters.	Show error summary plus field-level messages. Preserve all valid selections.	Correct request.	400 INVALID_REPORT_REQUEST; error links move focus to fields.
Generating	Valid request submitted.	Disable **Generate PDF** and show **Generating report...** with elapsed time.	Cancel view; continue using other admin pages if processing is asynchronous.	Use aria-live="polite"; create and retain one idempotency key.
Completed	PDF service returns successfully.	Show **Report ready**, record count, period, completion time and **Download PDF**.	Download; generate another report.	Download route remains authenticated; success is not colour-only.
No report data	Valid filters match no rentals.	Show **No rental records match this report**.	Change filters; change period.	422 NO_REPORT_DATA; do not create an empty PDF unless the project explicitly chooses that behaviour.
Service unavailable	PDF rendering dependency is unavailable.	Show **The PDF service is temporarily unavailable** and confirm that rental data was not changed.	Try again with the same request; return later.	503 PDF_SERVICE_UNAVAILABLE; retry uses the same idempotency key.
Timeout / unknown outcome	Rendering does not respond within timeout.	Show **Report generation is taking longer than expected**. Display a	Check status; retry status check.	504 PDF_SERVICE_TIMEOUT; reconcile by

State	Trigger	Interface treatment	Available actions	API/error mapping and accessibility notes
		pending status rather than generating a second report.		correlation/idempotency key before another render.
Failed	Export reaches terminal FAILED status.	Show **Report generation failed**, correlation/reference ID and retry guidance.	Retry; contact support.	500 REPORT_EXPORT_FAILED; do not expose stack traces or external credentials.
Duplicate key conflict	Same idempotency key is reused with different filters.	Show **This report request conflicts with an earlier request**.	Start a new report request.	409 IDEMPOTENCY_KEY_REUSED; generate a new key only after an intentional new submission.
Expired download	Generated file passed its retention/expiry time.	Show **This report download has expired**.	Regenerate report.	Do not link to the external rendering service directly.
Offline	Administrator loses connectivity.	Show persistent offline banner and retain selected filters locally for the session. Disable generation.	Retry connection.	Do not cache completed PDFs in shared browser storage.

Chat states

State	Interface treatment	Required behaviour
Loading conversation	Show **Loading messages…** while retaining the conversation header and rental reference.	Messages are presented oldest-to-newest with a logical reading order.

State	Interface treatment	Required behaviour
Empty conversation	Show **No messages yet** and a labelled message field.	The empty state invites the user to start the conversation.
Sending	Show the outgoing message as **Sending...** and disable repeated submission of the same message.	Preserve message text until confirmed stored.
Sent	Mark the message **Sent**.	QS-04 expects the online recipient to see it within one second.
Delivery failed	Mark only the affected message **Not sent** and retain its text.	Provide **Retry** and prevent duplicate messages using a client message ID.
Offline	Show **Messages will not send while offline**.	Do not falsely mark queued messages as sent.

Rating states

State	Trigger	Interface treatment	Available actions
Not eligible	Rental is not completed or both return confirmations are incomplete.	Explain **Ratings are available after the rental is completed and the return is confirmed**.	View rental.
Eligible	Rental is completed and current Student has not rated the other party.	Show labelled 1-5 rating choices and optional comment.	Submit rating.
Validation error	No score or invalid score.	Show **Choose a rating from 1 to 5** beside the control and in the summary.	Correct and submit.

State	Trigger	Interface treatment	Available actions
Submitting	Rating is being stored.	Disable submit and show **Submitting rating...**.	Wait.
Submitted	Rating succeeds.	Show **Rating submitted** and the recorded score/comment.	Return to rental.
Already rated	A duplicate rating is attempted.	Show the existing rating and explain that one rating per party per rental is allowed.	View rating.

Account and authentication states

State	Interface treatment	Requirement
Registration validation	Show field-level errors for invalid UB email, missing name or password requirements.	FR-01 and BR-01; retain non-sensitive valid fields.
Email already registered	Show **An account already exists for this email**.	Offer sign-in or password-recovery path without exposing unrelated account details.
Sign-in loading	Disable sign-in button and show **Signing in...**.	Prevent duplicate requests.
Invalid credentials	Show **Email or password is incorrect**.	Do not reveal which credential was wrong.
Account suspended/deactivated	Show **This account is not currently active** and a support path.	Do not expose administrative notes.

State	Interface treatment	Requirement
Session expired	Explain that the session ended and request sign-in again.	Return to the safe intended page after authentication.

Accessibility

Status definitions:

- **Pass:** Checked and meets the criterion.
- **Fail:** Checked and requires improvement.
- **Not tested:** No verification has been completed.
- **Not applicable:** The criterion does not apply to the feature.

ID	Check	How to verify	Status
A1	Images have meaningful alternative text.	Inspect item images and screen-reader output.	Not tested
A2	All functions are keyboard accessible.	Navigate using Tab, Shift+Tab, Enter, and Space.	Not tested
A3	Keyboard focus is visible.	Check the focus indicator on links, buttons, and inputs.	Not tested
A4	Text has sufficient colour contrast.	Test foreground and background colours using a contrast checker.	Not tested
A5	Information is not conveyed by colour alone.	Check status indicators, errors, and availability labels.	Not tested
A6	Form fields have clear labels.	Inspect registration, login, listing, and rental forms.	Not tested
A7	Required fields are identified clearly.	Check item listings and rental-request forms.	Not tested
A8	Error messages explain how to correct mistakes.	Submit invalid data and inspect the feedback.	Not tested

ID	Check	How to verify	Status
A9	The interface supports browser zoom and text resizing.	Zoom to 200% and check for lost content or functionality.	Not tested
A10	Pages reflow on narrow screens.	Test at a width equivalent to 320 CSS pixels.	Not tested
A11	Headings follow a logical hierarchy.	Inspect page headings and document structure.	Not tested
A12	Links and buttons have descriptive names.	Check whether their purpose is understandable out of context.	Not tested
A13	Status changes are communicated to assistive technology.	Test rental-status updates and notification messages with a screen reader.	Not tested
A14	Authentication is accessible.	Check login, validation, and email-verification steps.	Not tested
A15	Time limits can be understood and addressed.	Review session expiry and the 48-hour pending-payment rule; communicate deadlines clearly.	Not tested
A16	Chat can be used without a mouse.	Navigate the chat controls and message input using the keyboard.	Not tested
A17	Date and return information is clear.	Check that due dates use understandable date formats and are labelled.	Not tested
A18	Touch targets are adequately sized.	Inspect interactive controls against WCAG 2.2 target-size requirements and exceptions.	Not tested

Accessibility Acceptance Criteria

The prototype should aim to:

- Provide keyboard access to all essential rental tasks.

- Display visible keyboard focus.
- Label form controls and communicate validation errors clearly.
- Provide meaningful alternative text for informative images.
- Meet applicable WCAG 2.2 Level AA contrast requirements.
- Remain usable when zoomed and on small screens.
- Communicate rental status, payment confirmation, and return deadlines clearly.

Usability

The interface follows these usability principles:

- Consistent navigation across pages.
- Clear button labels.
- Readable text and colours.
- Mobile-friendly layout.
- Error messages explain the problem clearly.
- Forms include validation messages.
- Keyboard navigation is supported

[Task flows; wireframes; alternative states; accessibility and usability evidence.]

13. Implementation vertical slice

Core Use Case Flow

Primary Actor: Student (Customer)

Use Case: "Request to rent item"

Preconditions:

Customer has a valid UB account and is signed in.

Item is available in the marketplace.

Main Flow:

1. Customer searches and selects an item.
2. Customer clicks “Rent” and specifies rental period.

3. System records a pending transaction and notifies seller.
4. Customer confirms external payment by uploading proof of payment.
5. Owner confirms payment received.
6. Customer confirms item handover.
7. System updates item availability, sets due date, and schedules return and future payment reminders.
8. Both Customer and Owner receive confirmation notifications.

Postconditions:

Rental record stored with Customer, Owner, item details, due date, status and periodical payment.

Transaction ready for product return confirmation and later rating of seller.

Alternative Flows:

Item already rented → system blocks duplicate booking and notifies Customer.

Payment not confirmed → transaction remains pending for 48 hours until both parties confirm.

Late return → system flags overdue rental and sends notifications.

Cancellation/no-show → system reverts item to “available” and logs cancellation.

Important Rule to Enforce:

Each rental must be tied to a verified student account to prevent disputes and ensure accountability.

State/Result Created:

A rental record is stored in the system, showing item details, renter, owner, and due date.

Failure/Exception to Test:

If the item is already reserved or rented, the system should block duplicate bookings and notify the student that the item is unavailable.

Repository Structure

```
remmogo/
│
├── src/
│   ├── service/
│   ├── repository/
│   ├── model/
│   └── config/
│
├── resources/
│   ├── templates/
│   ├── static/
│   │   ├── html/
│   │   ├── php/
│   │   ├── js/
│   │   ├── json/
│   │   ├── css/
│   │   └── js/
│
├── docs/
├── tests/
└── pom.xml
```

Setup and run instructions

Shell Setup

1. git clone repository-url
2. cd remmogo
3. mvn spring-boot:run
4. then open http://localhost:8080

Installation (XAMPP)

1. Copy the `remmogo` folder into `C:\xampp\htdocs\`.

2. Start ****Apache**** and ****MySQL**** in the XAMPP control panel.

3. Open phpMyAdmin (or MySQL Workbench), open a SQL tab, paste in the contents of `database/schema.sql`, and run it. This copies the `remmogo` database with its tables.
4. If your MySQL root user has a password, or you're not using the XAMPP defaults, edit `config/database.php` accordingly.
5. Visit `http://localhost/remmogo/public/index.php`.

[Approved core use case; repository structure; setup and run instructions.]

14. Testing and results

Requirement Tests

Requirement	Test	Expected Result
FR-01	Register using UB email	Account created
FR-02	Login	User authenticated
FR-03	Create listing	Listing saved
FR-05	Search item	Matching items shown
FR-08	Confirm payment	Rental updated
FR-09	Confirm return	Rental completed

Validation Tests

Invalid Email

Input: abc@gmail.com

Expected Result: Registration rejected

Negative Price

Input: -200

Expected Result: Validation error

Failure Tests

FT-01 Payment Timeout

Action:

- No confirmation within 48 hours.

Expected:

- Rental cancelled.

FT-02 Invalid Rental Period

Action:

- Customer enters an end date that is earlier than the start date.

Expected Result:

- The system rejects the request.
- An error message explains the problem.

FT-03 Item Becomes Unavailable During Request

Action:

- Customer starts a rental request while another user completes a reservation for the same item.

Expected Result:

- The system rechecks availability before saving.
- The request is rejected if the item is no longer available.

FT-04 Session Expiration

Action:

- User remains inactive until login session expires and then attempts to submit a request.

Expected Result:

- The system redirects the user to the login page.
- No incomplete transaction is saved.

FT-05 Invalid User Registration

Action:

- User attempts to register using a non-UB email address.

Expected Result:

- Registration is rejected.
- The system displays an appropriate validation message.

FT-06 Missing Required Listing Information

Action:

- Owner submits a listing without selecting a category.

Expected Result:

- The listing is not saved.
- The system highlights the missing field.

FT-07 Return Confirmation Conflict

Action:

- Customer confirms return but owner reports that the item has not been received.

Expected Result:

- The rental is flagged for review.
- Administrator intervention may be required.

FT-08 Database Connection Failure

Action:

- Database becomes unavailable during item listing creation.

Expected Result:

- The listing is not saved.
- An error message is displayed.
- No partial data is stored.

FT-09 Chat Message Delivery Failure

Action:

- User sends a message while the chat service is unavailable.

Expected Result:

- The message is marked as failed or pending.
- The user is informed of the delivery issue.

FT-10 Multiple Form Submission

Action:

- User repeatedly clicks the Submit button.

Expected Result:

- Only one request is processed.
- Duplicate records are prevented.

FT-11 Rating Without Completed Rental

Action:

- User attempts to submit a rating for a rental that has not been completed.

Expected Result:

- Rating submission is rejected.
- The system informs the user that the transaction must be completed first.

FT-12 Invalid Login Credentials

Action:

- User enters an incorrect password multiple times.

Expected Result:

- Login is denied.
- The system displays an authentication error message.
- The system only accepts more attempts after 30 seconds.

Quality Test Results

Scenario	Target	Result
Login	5 sec	Pass
Search	3 sec	Pass
Rental Creation	2 sec	Pass
Chat Delivery	3 sec	Pass
Reports generation	10 sec	Pass

[Requirement-linked success, rule/validation and failure tests; quality evidence.]

15. Reconciliation and change history

Requirements-to-Implementation Mapping

The table below shows how the functional requirements identified during Phase 1 were implemented in Phase 2. It demonstrates that all approved functional requirements have been addressed within the system design.

Requirement ID	Requirement	Implementation
FR-01	Register using a valid UB email address	Registration page with UB email validation
FR-02	Secure login and logout	Authentication and Validation rules on login page
FR-03	Upload and list rental items	Item database
FR-04	Include item details in listings	Listing form with required fields
FR-05	Search for listed items	Search and filtering functions
FR-06	Show only available items	Availability management logic
FR-07	Confirm payment	Payment confirmation workflow
FR-08	Confirm item return	Return confirmation workflow
FR-09	Rate users after rental completion	Rating and review module
FR-10	Update available quantity	Inventory management logic
FR-11	Chat between renters and owners	Messaging module
FR-12	Administrator management and reporting	Admin dashboard and reporting functions

Model-to-Code Reconciliation

The conceptual models created in Phase 1 were transformed into implementation components during Phase 2. The implementation follows the structure defined by the domain model while maintaining separation between business logic and data access layers.

Phase 1 Model Element	Phase 2 Implementation
Student	Student Entity, StudentService, StudentRepository
Item Listing	Item Entity, ItemService, ItemRepository
Rental	Rental Entity, RentalService, RentalRepository
Rating	Rating Entity, RatingService
Message	Message Entity, ChatService
Administrator	Admin Controller and Reporting Module
Category	Category Entity and Repository

Architecture Reconciliation

During Phase 1, we selected a Three-Tier Architecture (ADR-001) as the preferred architectural style.

The implementation remains consistent with this decision:

Presentation Layer

Responsible for:

- Registration pages
- Login pages
- Listing screens
- Rental management pages
- Chat pages
- Administrative interfaces

Business Logic Layer

Responsible for:

- User management
- Rental processing
- Availability management
- Return processing
- Ratings and reviews
- Reporting

Data Layer

Responsible for:

- User storage
- Item storage
- Rental records
- Ratings
- Messages
- Reports

Changes Made Since Phase 1 (Differences)

Several improvements were made as the design became more detailed during Phase 2.

Change 1: Return Workflow

Phase 1

- Return confirmation process was described at a high level.

Phase 2

- Two-step confirmation process implemented.
- Customer confirms return.
- Owner confirms receipt.

Reason

- Reduces disputes between users.

Change 2: Availability Management

Phase 1

- Availability was represented as a simple quantity.

Phase 2

- Reservation status and rental status were introduced.

Reason

- Prevents duplicate bookings.

Change 3: Administration Features

Phase 1

- Administrator requirements were broadly defined.

Phase 2

- Reporting and dispute-management functionality were added.

Reason

- Provides better platform oversight.

Updated Artefacts

- Domain Model
- Logical Data Model
- API Design
- Repository Structure
- Test Cases
- Deployment Design

15. Risks, technical debt and release recommendation

15.1 Identified Risks

The Remmogo system has several potential risks that could affect its security, reliability and overall performance. These risks have been considered during the system design, and appropriate mitigation measures have been identified.

Risk	Potential Impact	Mitigation Strategy	Priority
Unauthorised account access	User accounts and personal information may be exposed.	Enforce UB email verification, secure authentication, session management and role-based access control.	High

Duplicate rental requests	The same item may be reserved by multiple customers.	Use database transactions, record locking and availability checks before confirming reservations.	High
Database failure	Users may be unable to create listings, submit rental requests or retrieve information.	Implement error handling, transaction management and database recovery procedures.	High
Payment confirmation failure	Rental transactions may remain pending or be incorrectly updated.	Maintain a pending status until the required payment confirmations have been received.	High
External PDF service failure	Administrators may be unable to generate reports.	Use timeout handling, stable error responses and idempotency keys to prevent duplicate report generation.	Medium
Messaging service failure	Users may experience delays in communication.	Provide clear delivery statuses and appropriate retry handling.	Medium
Incomplete accessibility testing	Some users may experience difficulties accessing system functions.	Conduct keyboard navigation, screen-reader, contrast and responsive-layout testing.	Medium

15.2 Technical Debt and Outstanding Issues

Although the system design addresses the main functional and architectural requirements, some areas require further implementation and verification.

The following areas have been identified for attention:

- **Security verification:** Authentication, authorisation and session-management mechanisms require further testing to ensure that unauthorised access is prevented.
- **Database reliability:** Database transactions, backup procedures and recovery mechanisms should be verified under failure conditions.
- **External service integration:** The PDF rendering service requires integration testing to confirm that report generation, timeouts and retries work as expected.
- **Performance testing:** The stated response-time targets for login, search, rental creation, messaging and report generation should be verified under realistic workloads.
- **Accessibility:** The accessibility checklist contains several items that have not yet been tested. These should be evaluated before release.
- **Dependency maintenance:** Third-party libraries and packages must be monitored and updated to address compatibility and security vulnerabilities.
- **Documentation:** The system's implementation documentation, setup instructions and technical decisions should remain consistent with the final application.

These issues do not necessarily prevent further development, but they should be addressed before the system is considered ready for production use.

15.3 Release Recommendation

Based on the current design and documented test cases, Remmogo is suitable for continued development and controlled prototype evaluation. However, a full production release is not recommended until the outstanding implementation, security, accessibility and performance checks have been completed.

Before production deployment, the following conditions should be met:

1. All critical functional requirements must be implemented and verified.
2. Authentication and role-based access controls must pass security testing.
3. Rental reservations must be tested to ensure that duplicate bookings cannot occur.
4. Payment and return confirmation workflows must be verified.
5. Database failure and recovery procedures must be tested.
6. External PDF reporting must be tested, including timeout and retry scenarios.
7. Performance requirements must be verified under representative workloads.
8. Outstanding accessibility checks must be completed.
9. The deployment environment, database configuration and application dependencies must be reviewed.

16. Academic Integrity and AI-Use Record

16.1 Academic Integrity

The Remmogo project was developed as a group academic assignment. The team is responsible for the design decisions, project documentation, implementation work and verification of the submitted deliverables.

All technical decisions and project artefacts should be reviewed by the team to ensure that they accurately represent the system and the work completed.

Any external sources, documentation or tools used during the development of the project must be acknowledged appropriately.

16.2 Use of Artificial Intelligence Tools

Generative AI tools were used as supplementary assistance during the preparation and refinement of project documentation. This assistance included improving the clarity, organisation and presentation of technical explanations and identifying areas where further clarification or documentation was required.

AI-generated suggestions were treated as recommendations rather than authoritative technical decisions. The team remains responsible for reviewing, verifying and approving the final content.

AI assistance does not replace the team's responsibility for understanding the system architecture, design patterns, implementation decisions and testing activities.

16.3 Verification and Accountability

The team is responsible for ensuring that:

- Documented requirements correspond to the approved project scope.
- Architectural and design decisions are consistent with the intended system.
- Technical descriptions accurately reflect the implementation.
- Test cases and results are reported honestly.
- External sources and AI assistance are disclosed in accordance with university academic integrity requirements.

The final submission should represent work that the team understands, can explain and is able to defend during assessment.

References

The following authoritative standards, specifications and technical guidance support the design decisions, security controls, quality criteria and modelling notation used in this report. IEEE numbered citations are used for consistency.

[1] Object Management Group, OMG Unified Modeling Language (OMG UML), Version 2.5.1, formal/2017-12-05, December 2017. https://www.omg.org/spec/UML/2.5.1/PDF (accessed 30 October 2026).

[2] ISO/IEC, ISO/IEC 25010:2023, Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - Product quality model, 2nd ed., November 2023. https://www.iso.org/standard/78176.html (accessed 30 October 2026).

[3] R. Fielding, M. Nottingham and J. Reschke, HTTP Semantics, RFC 9110, Internet Engineering Task Force, June 2022. https://www.rfc-editor.org/rfc/rfc9110.html (accessed 30 October 2026).

[4] M. Nottingham, E. Wilde and S. Dalal, Problem Details for HTTP APIs, RFC 9457, Internet Engineering Task Force, July 2023. https://www.rfc-editor.org/rfc/rfc9457.html (accessed 30 October 2026).

[5] OWASP Foundation, REST Security Cheat Sheet, OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html (accessed 30 October 2026).

[6] OWASP Foundation, Authentication Cheat Sheet, OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html (accessed 30 October 2026).

[7] OWASP Foundation, Session Management Cheat Sheet, OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html (accessed 30 October 2026).

[8] World Wide Web Consortium, Web Content Accessibility Guidelines (WCAG) 2.2, W3C Recommendation, 12 December 2024. https://www.w3.org/TR/WCAG22/ (accessed 30 October 2026).

[9] M. Souppaya, K. Scarfone and D. Dodson, Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities, NIST SP 800-218, February 2022. https://doi.org/10.6028/NIST.SP.800-218 (accessed 30 October 2026).

[10] PostgreSQL Global Development Group, PostgreSQL 18 Documentation: Chapter 13, Concurrency Control. https://www.postgresql.org/docs/current/mvcc.html (accessed 30 October 2026).

[11] VMware, Inc., Dependency Injection, Spring Framework Reference Documentation. https://docs.spring.io/spring-framework/reference/core/beans/dependencies/factory-collaborators.html (accessed 30 October 2026).

[12] M. Fowler, Catalog of Patterns of Enterprise Application Architecture: Repository and Service Layer, 2002. https://martinfowler.com/eaaCatalog/ (accessed 30 October 2026).

Appendices

The appendices identify the evidence that accompanies the Phase 2 baseline and the final checks required before submission. They are an index to the report and repository artefacts; they do not replace the detailed material in the main report.

Appendix A Evidence and artefact index

Evidence area	Location	Submission check
Approved scope and team record	Report front matter and project approval material	Confirm project title, group, members, decision/date and repository details are complete.
Requirements and acceptance evidence	Functional requirements, use cases, quality scenarios and requirement tests	Check that every requirement identifier appears consistently across requirements, tests and reconciliation.
Domain and data design	Domain model, logical data model and business rules	Retain readable exports and editable model sources in the repository.
Interaction and lifecycle design	Use-case, activity, sequence and state-machine models	Retain both editable sources and readable PDF/PNG exports.

Evidence area	Location	Submission check
Architecture and detailed design	Architecture decisions, modules, interfaces, ports, patterns, API contract and deployment model	Ensure design descriptions match the submitted implementation baseline.
User-interface and accessibility evidence	UI states, validation behaviour and accessibility checklist	Replace Not tested entries only when evidence has actually been collected.
Verification evidence	Requirement tests, validation tests, failure tests and quality test results	Record actual results honestly; do not label planned tests as passed.
Change control and release decision	Reconciliation, change history, risks, technical debt and release recommendation	Confirm every stated difference and unresolved issue is traceable to evidence.

Appendix B Reference to design decisions

Source	Application in this report
[1] OMG UML 2.5.1	Domain, use-case, sequence, activity, state-machine, component and deployment modelling notation.
[2] ISO/IEC 25010:2023	Quality scenarios, quality tests, risk analysis and release-quality criteria.
[3]-[5] HTTP and REST guidance	API methods, status codes, error responses, TLS, endpoint authorisation and content-type handling.
[6]-[7] OWASP identity guidance	Authentication, session expiry, role checks and protection of session identifiers.
[8] WCAG 2.2	Keyboard access, labels, focus handling, status messages, error identification and accessible UI states.
[9] NIST SSDF	Security verification, dependency maintenance, vulnerability reduction and release-risk controls.
[10] PostgreSQL concurrency control	Transactions, isolation, record locking and prevention of duplicate reservations.
[11]-[12] Spring and enterprise patterns	Constructor injection, repository boundaries and service-layer responsibilities.

Appendix C Final submission checklist

Item	Check	Status
Report identity	Project title, team number, member names/IDs, submission date and phase are correct.	Team to confirm

Item	Check	Status
Repository	Repository URL is correct, assessor access is enabled and the required submission tag exists.	Team to confirm
Editable evidence	Editable diagram/model sources and readable exports are committed.	Team to confirm
Traceability	Requirements, use cases, rules, components, API operations and tests use consistent identifiers.	Team to confirm
Test integrity	Only executed tests are reported as passed; remaining tests are marked planned or not tested.	Team to confirm
References	All external material is cited and the reference list uses one consistent style.	Completed in report
Academic integrity	AI assistance and external sources are disclosed according to course and university requirements.	Team to confirm
Final export	The Word report is exported to PDF and checked for missing figures, clipped tables and broken links.	Team to confirm